

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 Total Marks: 50

Annexure A

Design Task

Code of Conduct:

I Kanish Kumar S (192511415) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Kanish Kumar S

Student 12: Kanishkumar S | Register Number: 192511415

Design Task

Real-Time Emotion Detection System for Call Centers

Develop a distributed AI system using PySpark and RDD operations to analyze real-time audio and video streams from call center interactions for emotion detection. Describe the system architecture, the data fusion techniques used to combine voice-tone and facial-expression signals, and how intelligent agents coordinate to flag distressed customers and recommend de-escalation strategies to agents in real time.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration

Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

Real-Time Emotion Detection System for Call Centers

A Distributed AI Design using PySpark, RDD Operations & Intelligent Agents

1. Problem Understanding & Requirement Analysis

Call centers generate continuous, high-volume audio and video streams across thousands of simultaneous agent-customer sessions. The goal is to detect customer emotional distress in real time by jointly analysing voice tone and facial expression, and to trigger a coordinated agent response before the interaction escalates. This places four requirements on the design:

- Low latency: distress must be flagged within a few seconds of onset, not after the call ends.
- High throughput and horizontal scalability: the system must process thousands of concurrent streams.
- Multimodal fusion: voice-tone and facial-expression signals must be combined into one distress score.
- Actionable coordination: detection alone is not enough — intelligent agents must interpret the signal and recommend a concrete de-escalation strategy to the human agent.

PySpark's RDD/Structured Streaming model is a natural fit because it partitions incoming stream data across a cluster and applies the same transformation function in parallel to every partition — which is exactly the access pattern needed for per-session feature extraction and inference at scale.

2. Proposed System Architecture

The system is organised as an eight-stage distributed pipeline, shown below. Each stage maps to a Spark stage boundary or an agent process, allowing independent horizontal scaling.

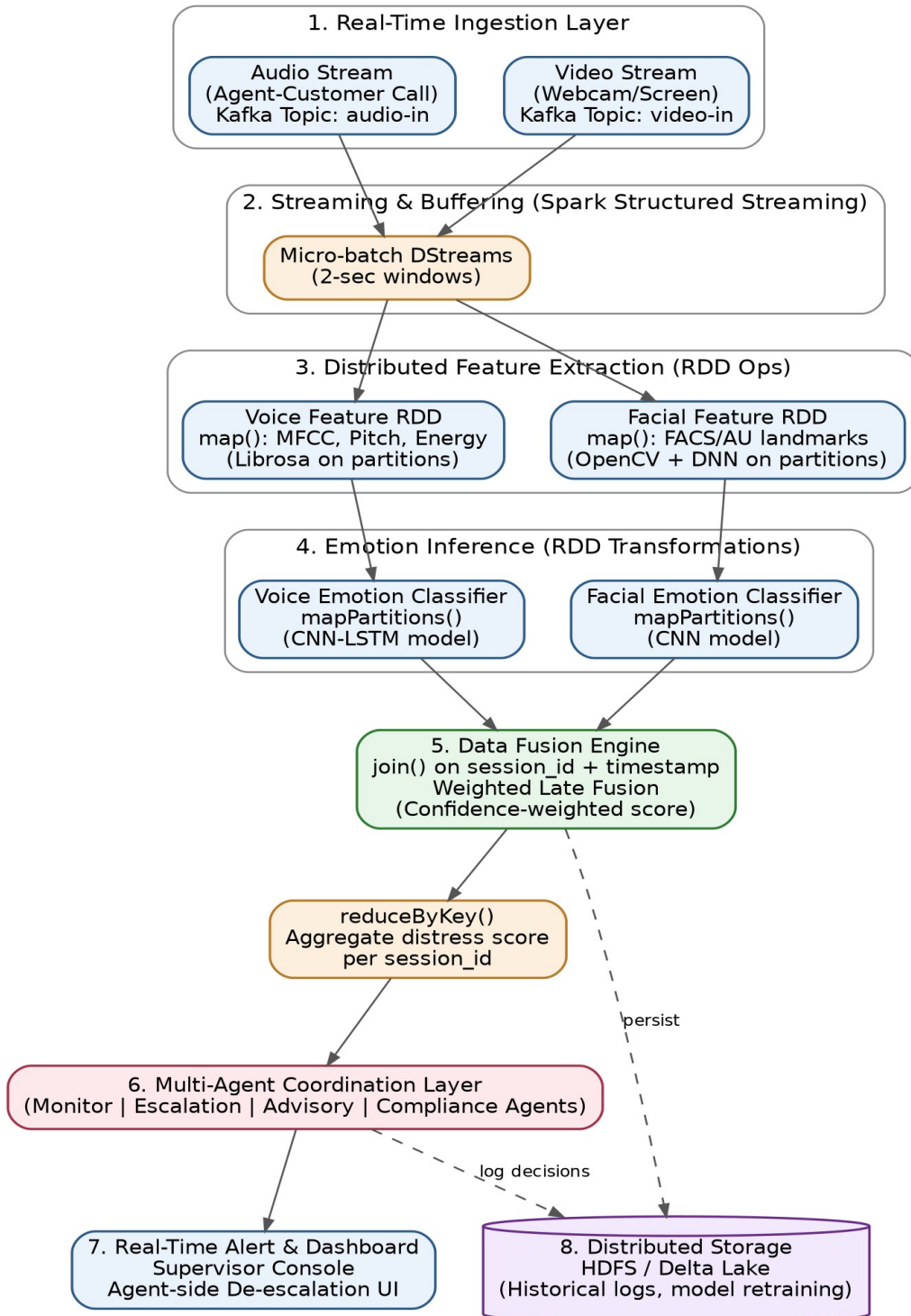


Figure 1: End-to-end distributed architecture for real-time emotion detection

2.1 Layer-by-Layer Description

- **Ingestion Layer:** Agent-customer audio is captured from the VoIP/telephony gateway; video is captured from the desktop or webcam feed (where consented). Both are published onto Kafka topics (audio-in, video-in) keyed by session_id, decoupling producers from the Spark cluster.

- **Streaming & Buffering:** Spark Structured Streaming consumes the Kafka topics in short micro-batches (~2-second windows), converting the continuous stream into a sequence of RDDs/DataFrames that downstream stages can process functionally.
- **Distributed Feature Extraction:** Each micro-batch RDD is partitioned by `session_id`. A `map()` transformation runs in parallel across the cluster's executors — Librosa-based signal processing extracts MFCCs, pitch, jitter, and energy from audio; OpenCV/DNN landmark models extract Facial Action Units (FACS) from video frames.
- **Emotion Inference:** `mapPartitions()` is used (rather than `map()`) so that the trained CNN-LSTM (voice) and CNN (facial) models are loaded once per partition/executor instead of once per record, which is essential for real-time throughput.
- **Data Fusion Engine:** Voice and facial emotion RDDs are joined on (`session_id`, timestamp window) using `join()`, and a weighted late-fusion score is computed (details in Section 3).
- **Multi-Agent Coordination Layer:** A set of cooperating intelligent agents consumes the fused score stream, decides whether to escalate, and selects a de-escalation strategy (Section 5).
- **Real-Time Alert & Dashboard:** Confirmed distress events are pushed to a supervisor console and to an in-call assist panel on the agent's screen.
- **Distributed Storage:** Fused scores and agent decisions are persisted to HDFS/Delta Lake for auditing, quality review, and periodic model retraining.

3. Data Fusion Techniques: Combining Voice-Tone and Facial-Expression Signals

Three fusion strategies were evaluated for combining the two modalities:

- **Early (feature-level) fusion** — concatenate raw voice and facial feature vectors before a single joint classifier. Rejected here because audio and video arrive at different sampling rates and can suffer independent packet loss, making a single synchronised feature vector fragile in a streaming setting.
- **Late (decision-level) fusion** — run independent classifiers per modality, then combine their output emotion probabilities. Adopted as the primary approach because it tolerates a missing or degraded modality (e.g., camera off) gracefully and keeps each model independently scalable and retrainable.
- **Hybrid fusion** — late fusion augmented with a small cross-attention layer over intermediate embeddings, used as an optional accuracy upgrade once the late-fusion baseline is stable in production.

The adopted weighted late-fusion score for a session at time window t is:

$$\text{distress_score}(t) = w_v * \text{conf}_v * P_{\text{voice}}(\text{distress} \mid t) + w_f * \text{conf}_f * P_{\text{face}}(\text{distress} \mid t)$$

where:

$P_{\text{voice}}(\text{distress} \mid t)$	= voice-model probability of a distress-related emotion class (anger, frustration, fear)
$P_{\text{face}}(\text{distress} \mid t)$	= facial-model probability of a distress-related expression (furrowed brow, frown, tension AUs)
$\text{conf}_v, \text{conf}_f$	= per-modality confidence (e.g. speech clarity, face-detection quality) in $[0,1]$
w_v, w_f	= tunable modality weights, $w_v + w_f = 1$

If a modality is unavailable in a given window (e.g., video dropped), its confidence `conf` falls to 0 and the fusion formula automatically re-normalises to rely on the remaining modality — this is the key practical benefit of late fusion over early fusion in a lossy streaming environment. In PySpark, the fusion step is implemented as a `join()` of the two per-modality RDDs on the (`session_id`, window) key, followed by a `map()` that applies the weighted formula above.

4. PySpark & RDD Operations Design

The pipeline is expressed almost entirely through RDD transformations (lazy) and actions (triggering execution), which is what gives it horizontal scalability. A representative skeleton:

```
# 1. Ingest micro-batch of audio & video chunks from Kafka
audio_rdd = ssc.kafkaStream('audio-in').map(parse_audio_chunk)
video_rdd = ssc.kafkaStream('video-in').map(parse_video_frame)

# 2. Distributed feature extraction (map: 1 record -> 1 record)
voice_features = audio_rdd.map(extract_voice_features) # MFCC, pitch, energy
face_features = video_rdd.map(extract_facial_landmarks) # FACS Action Units

# 3. Batched model inference (mapPartitions: load model once per partition)
voice_emotion = voice_features.mapPartitions(run_voice_emotion_model)
face_emotion = face_features.mapPartitions(run_facial_emotion_model)

# 4. Key by (session_id, window) and fuse modalities
voice_kv = voice_emotion.map(lambda r: ((r.session_id, r.window), r))
face_kv = face_emotion.map(lambda r: ((r.session_id, r.window), r))
fused = voice_kv.join(face_kv).map(compute_weighted_fusion_score)

# 5. Aggregate a rolling distress score per session
session_scores = fused.map(lambda r: (r.session_id, r.score)) \
    .reduceByKey(lambda a, b: max(a, b))

# 6. Filter sessions crossing the distress threshold and dispatch to agents
flagged = session_scores.filter(lambda kv: kv[1] >= DISTRESS_THRESHOLD)
flagged.foreachPartition(dispatch_to_agent_layer)
```

Key RDD operations and why each is used:

- `map()` — stateless, per-record feature extraction; fully parallelisable across partitions with no shuffle.
- `mapPartitions()` — amortises expensive model-loading cost across a whole partition instead of every record, critical for real-time latency.
- `join()` — synchronises the two modality streams on a common (`session_id`, `window`) key to perform fusion.
- `reduceByKey()` — cheaply aggregates the rolling maximum/weighted-average distress score per session without collecting data to the driver.
- `filter()` and `foreachPartition()` — select only sessions that cross the distress threshold and dispatch them directly from each executor to the agent layer, avoiding a driver bottleneck.

Fault tolerance comes from RDD lineage: if an executor fails mid-batch, Spark recomputes only the lost partitions from the last checkpoint rather than losing the whole streaming job — essential given the system runs continuously, 24/7.

5. Intelligent Agent Coordination

Emotion scores by themselves do not tell an agent what to do. A layer of cooperating intelligent agents interprets the fused signal and turns it into an actionable recommendation. Four specialised agents coordinate as a pipeline, each reacting to events published by the previous one (a lightweight blackboard/event-bus pattern):

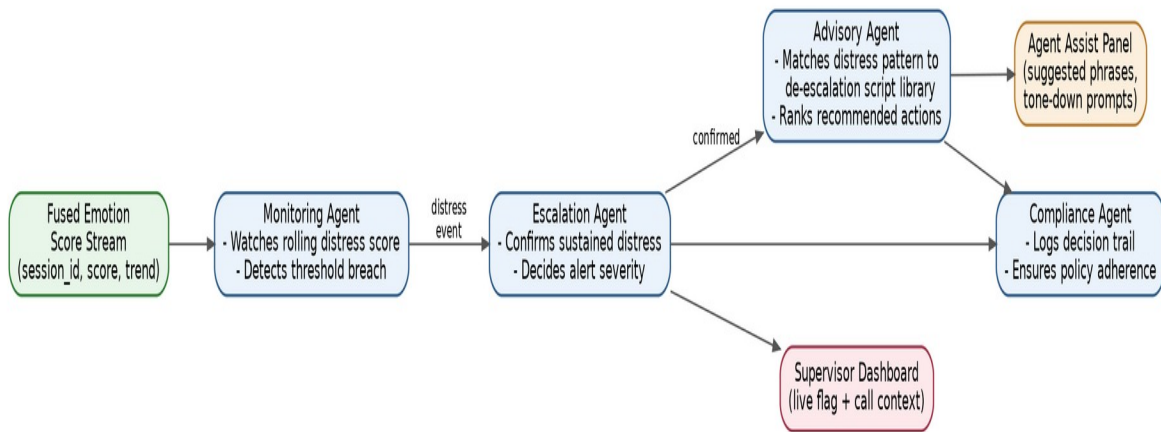


Figure 2: Multi-agent coordination pipeline for distress escalation and advisory

- **Monitoring Agent:** continuously watches the rolling `distress_score(t)` for every active session and raises a 'distress event' the moment the score crosses the configured threshold for a sustained window (avoiding false positives from a single noisy frame).
- **Escalation Agent:** confirms the event is sustained (not a transient spike), assigns a severity level (Low/Medium/High), and decides whether supervisor visibility is required.
- **Advisory Agent:** matches the specific distress pattern (e.g., rising vocal pitch + furrowed brow = frustration; flat tone + tearful expression = sadness) against a library of de-escalation scripts, and ranks the top recommended phrases/actions for the human agent.
- **Compliance Agent:** logs every decision, the evidence behind it, and the recommendation issued, both for QA review and for regulatory audit trails.

The agents coordinate asynchronously through the same event stream that carries the fused score — each agent subscribes only to the events relevant to its role, so agents can be scaled, retrained, or replaced independently without redesigning the pipeline.

6. Real-Time De-escalation Recommendation Workflow

Once the Advisory Agent selects a recommendation, it is pushed to two destinations simultaneously: an in-call assist panel on the agent's screen (e.g., "Customer frustration rising — acknowledge concern, lower pace, offer concrete next step") and, for High-severity cases, a live flag on the supervisor's dashboard so a team lead can silently join or intervene. Because the recommendation must reach the agent within the same call, the entire chain — feature extraction, inference, fusion, agent reasoning, and UI delivery — is budgeted to complete within roughly 2-4 seconds end-to-end.

7. Scalability & Distributed Computing Design

- **Horizontal scaling:** adding executors linearly increases the number of concurrent sessions the cluster can process, since each session's processing is independent and keyed by `session_id`.
- **Partitioning strategy:** streams are partitioned by `session_id` so that all data for one call stays on one executor, avoiding unnecessary shuffles during feature extraction.
- **Backpressure handling:** Spark Structured Streaming's built-in backpressure throttles ingestion if executors fall behind, preventing memory overload during call-volume spikes (e.g., during an outage affecting many customers at once).

- Fault tolerance: RDD lineage plus periodic checkpointing to HDFS allows the streaming job to recover from executor or node failure without losing in-flight sessions.
- Elastic resource management: the cluster runs under YARN/Kubernetes dynamic allocation so executor count grows during peak call-center hours and shrinks off-peak.

8. Data Flow & Processing Pipeline Summary

End-to-end, the pipeline can be summarised as a single directed flow: Audio/Video capture → Kafka ingestion → Spark micro-batching → parallel feature extraction (map) → parallel emotion inference (mapPartitions) → cross-modal fusion (join) → session-level aggregation (reduceByKey) → threshold filtering (filter) → multi-agent reasoning → real-time dispatch to agent UI and supervisor dashboard → persistent audit log (HDFS/Delta Lake). Every arrow in this chain is either a Spark transformation, a Spark action, or an asynchronous agent-to-agent event, which keeps the whole system stateless and horizontally scalable at each stage.

9. Innovation & Practical Impact

- Confidence-weighted late fusion degrades gracefully when a modality is missing, rather than failing outright — important for real call-center conditions where video is not always available or consented.
- The role-specialised agent pipeline (Monitor → Escalate → Advise → Comply) mirrors how a human QA/supervisor team actually operates, making the system's recommendations interpretable and auditable rather than a black-box alert.
- Because feature extraction and inference are expressed purely as RDD map/mapPartitions transformations, the same pipeline code can run unmodified on a single laptop for testing or a 500-node cluster in production, differing only in the number of partitions.

10. Conclusion

The proposed design demonstrates a scalable, fault-tolerant distributed AI system built on PySpark and RDD operations that fuses voice-tone and facial-expression signals through confidence-weighted late fusion, and hands off detected distress to a coordinated multi-agent layer that turns raw emotion scores into concrete, real-time de-escalation guidance for call-center agents — meeting the low-latency, high-throughput, and actionability requirements identified at the outset.